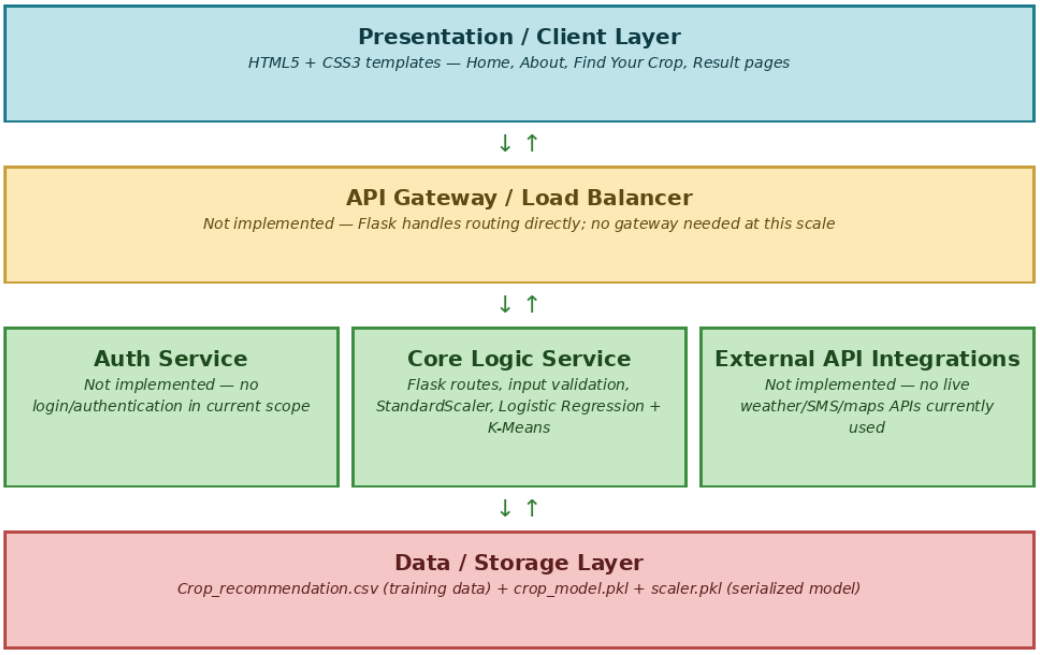
Solution Architecture

Date	
Team ID	
Project Name	Smart Agricultural Production Optimization Engine (OptiCrop)
Maximum Marks	5 Marks

Solution Architecture Diagram:

This diagram maps OptiCrop's actual architecture onto the Presentation, Application, and Data layer structure. Layers that are not part of the current implementation (API Gateway, Auth Service, External API Integrations) are explicitly marked as not implemented, rather than filled in with technologies that were not actually used.



Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the Home, About, Find Your Crop, and Result pages. Collects the 7 soil/climate input values from the user and displays the predicted crop after submission.	HTML5, CSS3 (custom-styled, no JS framework)
API Gateway	*Not implemented. Flask's built-in routing handles all requests directly (/ , /about, /findyourcrop, /predict) — a separate gateway or load balancer is unnecessary at this project's scale (single instance, low traffic).*	*Not applicable*

Core Logic / Microservices	A single Flask application (not split into microservices) that validates form input, scales it using the fitted StandardScaler, and runs the Logistic Regression model to predict the recommended crop. K-Means clustering is used separately for exploratory analysis, not live prediction.	Python, Flask, Scikit-learn (LogisticRegression, KMeans, StandardScaler), NumPy
Database	Stores the training dataset and the serialized trained model. No database server is used — the CSV is read at training time, and the model/scaler are loaded into memory from disk when the Flask app starts.	CSV file (Crop_recommendation.csv), Pickle (crop_model.pkl, scaler.pkl)

Note: Auth Service and External API Integrations shown in the diagram template are not part of the current build — there is no user login and no external (weather, payment, email, etc.) API calls in this version of OptiCrop.